

TOLNA VÁRMEGYEI SZC Apáczai Csere
János Technikum és Kollégium

Vizsgaremek

Készítették:

Timár Dávid

Hederics Nikolasz György

Lódri Gergő

Pécs

2025

TOLNA VÁRMEGYEI SZC Apáczai Csere János Technikum és Kollégium

A szakma megnevezése: **Szoftverfejlesztő és -tesztelő**

A szakma azonosító száma: **5 0613 12 03**

Vizsgaremek

InnovIT

Készítették:

Timár Dávid

Hederics Nikolasz György

Lódri Gergő

Pécs

2025

Tartalomjegyzék

Feladat rövid ismertetése.....	5
Bevezetés	5
Cél	5
Jövőbeli tervek	5
Használt technológiák.....	6
Programozási nyelvek	6
Adatbázis típusa	6
Egyéb technológiák.....	6
Adatbázis.....	7
Adatbázis rövid magyarázata	7
Fejlesztés módszertan	9
Scrum	9
Elvégzett tesztek	10
Tesztelési fajták.....	10
Jelentősége.....	11
Használt technológiák.....	11
Oldal működésének részletes leírása	12
Az oldal részletes leírása	12
Adminként bejelentkezve.....	12
Felhasználóként bejelentkezve.....	13
Fejlesztői dokumentáció.....	14
Fejlesztési technológiák	14
HTTP protokollok	14
A rétegek szerepköre a projektben.....	15
A projektben való rétegek felépítése	17
Config Package.....	17
Controller Package	17

Modell Package	17
Service Package	18
<i>A frontend részletes leírása</i>	<i>18</i>
Könyvtárstruktúra	18
Komponensek	19
Stíluslapok	19
Reszponzív design	19
AuthService	20
HTTP kérések	20
Modulok és szolgáltatások	20
A frontend egyes részein lévő főbb funckiók	21
Lokális szerver futtatása	22
<i>Csapatmunka</i>	<i>22</i>
Szerepek	22
<i>Jövőbeli terveink</i>	<i>25</i>

Feladat rövid ismertetése

Bevezetés

Ez a dokumentáció áttekintést nyújt az autókölcsönzőnk által kínált szolgáltatásokról, a bérleti feltételekről és a szükséges eljárásokról és ahhoz, hogy egy zökkenőmentes, kényelmes szolgáltatást tudjunk nyújtani az ügyfeleknek. A dokumentum célja, hogy részletes tájékoztatást nyújtson ügyfeleink számára az autóbérlés folyamatáról és a kapcsolódó tudnivalókról.

Cél

Elsődleges célunk, hogy megbízható és minőségi járműveket biztosítsunk ügyfeleink számára, ezzel segítve őket a mobilitási igényeik kielégítésében a szigeten. Célunk, hogy a sziget egyik vezető autókölcsönzőjévé váljunk, ehhez pedig kiváló ügyfélszolgálat és versenyképes árakat kínálunk. Szeretnénk, ha ügyfeleink elégedetten távoznának tőlünk, ezért a bérleti folyamatot a lehető legkényelmesebbé és átláthatóbbá tesszük. Fontosnak tartjuk, hogy széleskörű járműparkkal rendelkezünk, így mindenki megtalálhassa a számára leginkább megfelelő autót.

Jövőbeli tervek

A jövőben tervezzük az elektromos robogók és kerékpárok bérleti lehetőségének bevezetését is, így a rövidebb távolságok megtétele is környezetbarát és kényelmes lehet a szigeten. Szeretnénk együttműködések kialakítani a sziget helyi szálláshelyeivel és turisztikai látványosságával, hogy a vendégek számára kedvezményes csomagokat és integrált mobilitási megoldásokat kínálhassunk. A későbbiekben szeretnénk online fizetést kínálni az ügyfeleinknek, hogy otthonról gondmentesen tudjanak fizetni. Terveink között szerepel egy üzenetküldő rendszer, ami emailben tájékoztatja az ügyfeleket, hogy pontosan mikor kell felvenni és leadni az járművet. Hosszú távon gondolkodunk töltőállomások kiépítésében a sziget különböző pontjain, ezzel is támogatva az elektromos járművek használatát és a fenntartható turizmust.

Használt technológiák

Programozási nyelvek

A projekt során különböző programozási nyelveket alkalmaztunk a rendszer különböző részein:

1. Frontend
 - HTML
 - CSS
 - TypeScript
 - Angular (keretrendszer)
2. Backend
 - Java (OOP)

Adatbázis típusa

Az adatbázisunk hatékonyan működő relációs adatbázis, MySQL alapokon. A relációs adatbázisok strukturált és könnyen kezelhető adatokat biztosítanak, és az MySQL megbízható keretek között teszi lehetővé azok hatékony kezelését.

Egyéb technológiák

1. Git
 - A Git egy elosztott verziókezelő rendszer, amely segíti a fejlesztőket a kódbázis nyomon követésében, változtatások kezelésében és együttműködésben a csapaton belül. Az elosztott jellege lehetővé teszi a párhuzamos fejlesztést, valamint a könnyű visszatérési pontok (commit) készítését és visszaállítását
2. Jira
 - A Jira egy olyan project management eszköz, amely lehetővé teszi a csapatoknak a projektjeik nyomon követését, feladatok kezelését és a fejlesztési folyamatok hatékony szervezését. Az eszköz segíti a feladatok rendszerezését, a prioritások meghatározását és a csapat tagjainak közötti kommunikációt.

3. Figma

- Egy webes design- és prototípuskészítő eszköz, amely lehetővé teszi a tervezők és fejlesztők számára, hogy együtt dolgozzanak a felhasználói felületek (UI) tervezése és prototípusok készítése során. A Figma segítségével a csapat könnyen együttműködhet, valós időben megoszthatja és szerkesztheti a tervezéseket, így elősegítve a hatékony kommunikációt és a tervezési folyamatok szervezését.

4. Discord

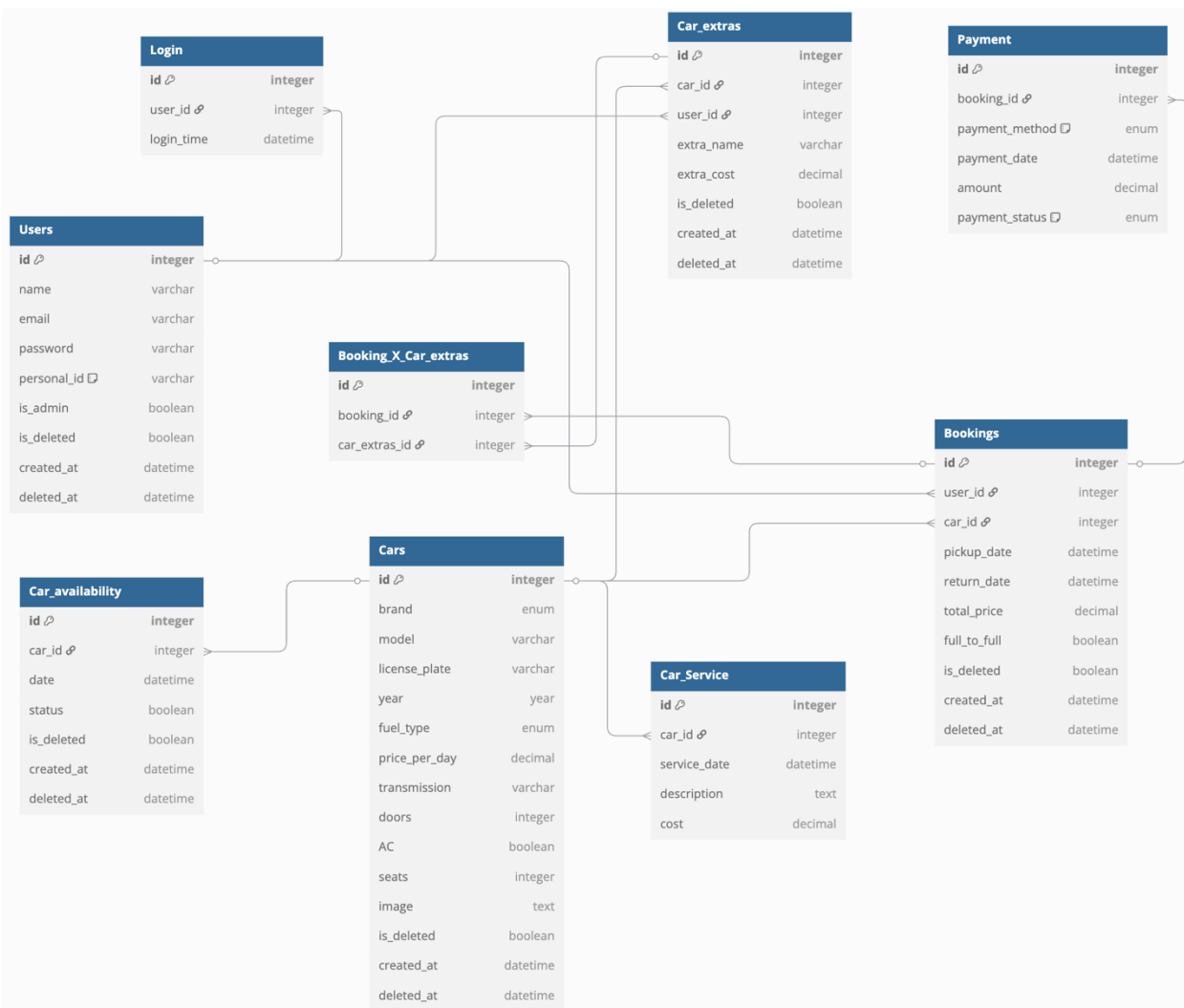
- A discord lehetővé tette a csapat számára, hogy hatékonyan tudjon kommunikálni és mindenki nyomon tudta követni a projektben lévő változásokat.

Adatbázis

Adatbázis rövid magyarázata

Az alkalmazás optimális működéséhez elengedhetetlen, hogy minden olyan információt, amelyre az alkalmazásnak szüksége van, megfelelően és hatékonyan tároljunk. Ennek érdekében kiemelkedő jelentősége van az adatbázis pontos megtervezésének, hiszen ez a webalkalmazás stabilitásának és hatékonyságának alapvető építőköve. Az adatok strukturált és szervezett tárolása nemcsak a felhasználói élményt javítja, hanem lehetővé teszi az alkalmazás számára, hogy gyorsan és hatékonyan hozzáférjen és kezelje ezeket az adatokat.

Az objektumorientált programozás (OOP) integrációja a backend oldalon kritikus fontosságú ezen célok elérésében. Az OOP megközelítés lehetővé teszi, hogy minden modell osztály egyértelműen és konzisztensen reprezentálja a rendszerben szereplő egyedi entitásokat. Minden osztály egy-egy egyedet képvisel, és a kapcsolatokat más osztályokkal való egyszerűbb és átláthatóbb módon modellezi. Ezáltal az alkalmazás backend része könnyebben karbantartható, bővíthető és skálázható lesz.



A képen a rendszerünk adatbázis sémájának ER diagramja (Entity-Relationship diagram) látható, ami az adatbázisban tárolt objektumok (entitások) és azok kapcsolatainak vizuális megjelenítése. A diagram elemei között táblák (entitások), attribútumok és kapcsolatok (relációk) találhatók. Az egyes táblák elsődleges kulcsait (Primary Key) és idegen kulcsait (Foreign Key) is jelölik.

A diagram több fő részre osztható, mindegyik különböző funkcionális területeket képvisel:

1. **Felhasználói adatok:** A `users` tábla tárolja a felhasználók alapadatait, mint név, email, személyi igazolvány és egyéb elérhetőségi adatok. Kapcsolódik a `bookings`, `login`, és `car_extras` táblákhoz, amelyek részletezik a felhasználók foglalásait, bejelentkezési adatait és a

pluszként lefoglalt extrákat az autóhoz. Az idegen kulcsok segítségével biztosítják az adatok integritását és kapcsolatukat.

2. **Autókkal kapcsolatos információk:** A `cars`, `car_availability`, `car_extras`, `car_service` táblák kezelik az autókhoz kapcsolatos tevékenységi adatokat. A `cars` tábla az autókhoz kapcsolatos információkat tárolja, míg a `car_availability` tábla az autók elérhetőségét tárolja. A `car_extras` tábla az autókhoz rendelhető extrákat tárolja, míg a `car_service` tábla az autókhoz kapcsolatos szervízeket tárolja.
3. **Foglalásokkal kapcsolatos adatok:** A `bookings` tábla az adott userhez tartozó foglalásokat tárolja és azt hogy melyik autót foglalta le
4. **Fizetéssel kapcsolatos információk:** A `payment` tábla a fizetéseket kezeli, ami kapcsolódik az adott foglalást végző userhez

A diagram komplex kapcsolati struktúráját és az adatintegritást idegen kulcsok biztosítják, amelyek garantálják, hogy az összekapcsolt adatok konzisztensek és naprakészek maradjanak. Az ER diagram átfogó képet nyújt arról, hogyan kapcsolódnak egymáshoz az adatok, és hogyan kezelhetők hatékonyan az iskolai rendszerben.

Fejlesztés módszertan

Scrum

1. Rugalmasság a változásokra
 - A Scrum módszertan lehetővé tette számunkra, hogy rugalmasan alkalmazkodjunk a változásokhoz a projekt folyamán. Az iteratív fejlesztési ciklusok és a rendszeres retrospektív értekezletek révén könnyen integráltuk a visszajelzéseket, és gyorsan reagálhattunk az esetleges változásokra, optimalizálva ezzel a projekt irányát és funkcionalitását.
2. Rendszeres fejlesztése ciklusok
 - A Scrum keretrendszer rendszeres, általában két-három hetes időszakokra tervezett fejlesztési ciklusokat biztosított számunkra. Ez lehetővé tette, hogy gyorsan és rendszeresen hozzáférjünk a fejlesztett funkcionalitásokhoz, és azonnali visszajelzést kapjunk a csapat és az érintettek részéről.

3. Csapatmunka és átláthatóság

- A Scrum keretrendszer elősegítette a hatékony csapatmunkát és átláthatóságot a projektben. A rendszeres Scrum értekezletek és a projekt backlog frissítése révén minden csapattag számára világos volt a projekten belüli állapot és prioritások.

Elvégzett tesztek

Tesztelési fajták

Tesztelés során két alapvető típust különböztetünk meg: manuális és automatizált (Unit) tesztelés. Mindkét típusnak saját jelentősége és szerepe van az alkalmazás minőségének biztosításában.

1. Manuális tesztelés

- A manuális tesztelés az emberi interakcióra épít, a tesztelő kézzel végzi el a teszteket. Ez a fajta tesztelés különösen fontos az alkalmazás felhasználói élményének validálásában és a funkciók intuitív használatának biztosításában. A manuális tesztelés során a tesztelő manuálisan interagál az alkalmazással, ellenőrzi a felhasználói felületet, és szimulálja a valós felhasználói környezetet.

2. Selenium tesztelés

- A Selenium egy népszerű nyílt forráskódú tesztelőeszköz, amelyet webalkalmazások tesztelésére használnak. Lehetővé teszi a tesztelők számára, hogy automatizált teszteket írjanak különféle programozási nyelveken, hogy teszteljék a webalkalmazások működését. A szeléntesztek számos különböző böngészőn és operációs rendszeren futtathatók.
- A főbb funkciók seleniumban lettek letesztelve

3. Automatizált (Unit) tesztelés

- A unit tesztelés a kód kisebb egységeit, például funkciókat vagy metódusokat vizsgálja izoláltan. Ennek a tesztelési típusnak a fő célja a kód működésének megerősítése, valamint a hibák korai felfedezése és javítása. A unit tesztek automatizáltak és ismételhetők, így segítik a gyors fejlesztést és a kód stabil állapotának fenntartását.

Jelentősége

1. Manuális tesztelés

- Az egyik kulcsfontosságú szempont a felhasználói élmény ellenőrzése. A tesztelők vizsgálják, hogy az alkalmazás könnyen kezelhető-e, érthető-e a felhasználók számára, és az általános használat során hogyan reagál a felhasználói interakciókra.
- A másik fontos szempont a funkcionális validáció, amely biztosítja, hogy az alkalmazás különböző funkciói a tervezett módon és specifikációknak megfelelően működjenek. A tesztelők ellenőrzik a különböző funkciókat, scenáriókat és bemeneteket annak érdekében, hogy az alkalmazás minden szempontból megfeleljen a tervezésnek.

2. Selenium tesztelés

- A manuális tesztelés időigényes és ismétlődő feladat lehet. A Selenium segítségével a tesztesetek automatizálhatók, így jelentősen csökkenthető a tesztelésre fordított idő.
- Az automatizált tesztek gyorsabban és gyakrabban futtathatók, ami lehetővé teszi a webalkalmazás több funkciójának és forgatókönyvének a lefedését.

3. Automatizált (Unit) tesztelés

- A unit tesztek kritikus szerepet játszanak a kód stabilitásának és megbízhatóságának fenntartásában. A tesztek segítenek abban, hogy az egyes kódmodulok a tervezett módon működjenek, és az esetleges változások vagy fejlesztések során ne okozzanak nem várt hibákat a meglévő funkcionalitásban.
- A unit tesztelés korai hibafeltárássá összpontosít. Mivel a tesztek automatizáltak és ismételhetők, így lehetőséget teremtenek a fejlesztők számára a gyorsabb hibajavításra.

Használt technológiák

- Manuális tesztelés → Chrome
- Automatizált (Unit) tesztelés → Junit (Java)

Oldal működésének részletes leírása

Az oldal részletes leírása

Az oldal két fő felhasználói profilra összpontosít

- Felhasználó
- Admin

Az alkalmazás bejelentkezési folyamata során a felhasználó jogosultságait figyelembe véve irányítja át az adott felhasználót az alkalmazás különböző részeire.

Adminként bejelentkezve

Ez a felület lehetővé teszi az adminok számára, hogy intuitív és könnyen kezelhető módon hozzáférjenek azokhoz az információkhoz, amelyekre az autókölcsönző során az adatok optimalizálása és kézben tartásához szükségesek.

Főbb funkciók:

1. Felhasználók kilistázása
 - Itt az adminnak lehetősége van kilistázni az összes felhasználót és azoknak az összes adatát. Ennek révén az adminok könnyen hozzá tudnak férni az adott felhasználó adataihoz
2. Adminok kilistázása
 - Ugyan úgy, mint a felhasználóknál, kilistázza az adminok adatait és ezzel betekintést nyer az admin, hogy kik rendelkeznek ezzel a joggal.
3. Foglalások kilistázása
 - Itt az adminnak lehetősége van kilistázni az összes eddigi foglalásokat, ami visszaadja a foglaláshoz tartozó adatokat.
4. Felhasználók, adminok, foglalások frissítése

- Itt az adminnak lehetősége van a felhasználókat, adminokat, foglalásokat frissíteni szükség szerint.
5. Felhasználók, adminok, foglalások törlése
 - Az adminnak engedélye van felhasználókat, adminokat (logikai törlések) és foglalásokat törölni szükség szerint.
 6. Autó hozzáadása, frissítése, törlése
 - Az adminnak lehetősége van egy új autót hozzáadni, ha szeretné bővíteni a garázsát. Ugyan itt lehetősége van egy autót frissíteni vagy törölni, attól függően, hogyan változik az adott autó.

Felhasználóként bejelentkezve

Amikor a felhasználók bejelentkeznek a rendszerbe, egy átgondolt és könnyen kezelhető felhasználói felület fogadja őket. Az interaktív rendszer lehetővé teszi a felhasználók számára, hogy egyszerűen és hatékonyan tudják használni az oldalt, biztosítva az átláthatóságot és a kényelmet.

Főbb funkciói:

1. Autók kilistázása az adott időpontra
 - A felhasználó ki tudja választani, hogy milyen időpontban szeretne autót bérelni és a rendszer kilistázza azokat az autókat, amik az adott időpontban elérhetőek. Ez lehetővé teszi a gyors autófoglalást igény szerint.
2. Szűrés autók között
 - A `Cars` oldalon a felhasználóknak lehetőségük van szűrni az adott autók közül. A felhasználók számos szűrő közül választhat, mint pl.: márka, férőhely, ár, évjárat és üzemanyag. Ez megkönnyíti az autók kiválasztását a felhasználói igényeknek megfelelően. Emellett van egy reset filter gomb, ami visszateszi az eredeti pozícióba a szűrőt, azaz nincs semmi kijelölve.

3. Autó lefoglalása

- Ha a felhasználó megtalálta a számára megfelelő autót akkor azt le is tudja foglalni a reserve gombra kattintva, ami átirányítja a felhasználót a reservation oldalra, ahol meg tudja tekinteni az autó részleteit és megerősíteni a foglalást.

4. Profilkép beállítása

- A felhasználó a bejelentkezés után be tud állítani magának egy profilképet, ha szeretne, ami fokozza a felhasználói élményt.

5. HA van felhasználói jogosultság

- Ha van olyan felhasználói jogosultsága, akkor lehetősége van új autót berakni, vagy éppen kivenni, esetlegesen szerkeszteni egy autót.

Fejlesztői dokumentáció

Fejlesztési technológiák

A fejlesztés során az Angular keretrendszert választottuk frontend részre, biztosítva a felhasználóbarát és dinamikus felületet. A backend implementációhoz pedig a JAX-RS REST API-t használtuk, kiemelve annak standard Java EE specifikációját, könnyű konfigurálhatóságát annotációkkal, keretrendszerfüggetlenségét és könnyű integrálhatóságát más Java EE technológiákkal.

Az alkalmazás alapja egy relációs adatbázisrendszer, MySQL. A relációs adatbázis lehetővé teszi az adatok hatékony tárolását és kezelését, biztosítva az alkalmazás stabilitását és adatbiztonságát.

HTTP protokollok

A szerver-kliens kommunikáció során a kérések között négy alapvető HTTP módszert alkalmaztunk:

1. GET

- Olvasási műveletekhez használt, könnyen tesztelhető és a paraméterek az URL-ben megoszthatók.

- Gyakran alkalmazva információ lekérésére, a válaszok gyorsítótár használatával optimalizálhatók.

2. POST

- Módszer írási műveletekhez ideális, biztonságos, és a kérés testében küldi az adatokat, megfelelő az állapotváltozások kezelésére.
- A POST kérés adatai a kérés testében vannak, nem az URL-ben, így alkalmasabb érzékeny információk küldésére.
- Használatos adatok küldésére a szerver felé, például űrlapok kitöltésekor vagy fájlok feltöltésekor.

3. PUT

- Meglévő erőforrások frissítésére szolgál.
- Ha egy meglévő erőforrás (például egy felhasználói profil,) bizonyos adatait módosítani kell, a PUT kérés elküldi a teljes frissített reprezentációt a szervernek. A szerver ezután felülírja a régi adatokat az újakkal.
- A PUT kérés a projekt fejlesztése során elengedhetetlen az adatokkal való interakcióhoz, lehetővé téve a meglévő információk megbízható és következetes frissítését, valamint bizonyos esetekben új erőforrások létrehozását a szerveren.

4. DELETE

- Meglévő erőforrások törlésére szolgál a szerveren.
- Ha egy adott erőforrást (például egy felhasználói fiókot, egy autót, egy foglalást) el kell távolítani a szerverről, a DELETE kérés elküldi az erőforrás egyedi azonosítóját (URI-ját). A szerver ezután törli a hozzá tartozó adatokat.
- a DELETE kérés a projekt fejlesztése során nélkülözhetetlen a feleslegessé vált vagy már nem szükséges adatok és erőforrások eltávolításához a szerverről, biztosítva ezzel az adatbázis tisztaságát és a rendszer hatékony működését.

A rétegek szerepköre a projektben

Config Package

Szerep: A Config package felelős az alkalmazás konfigurációs beállításainak kezeléséért.

Funkciók:

- **Konfigurációkezelés:** A Config package tartalmazza azokat az osztályokat, amelyek az alkalmazás konfigurációjának kezelését végzik, például a CORS beállítások, adatbázis kapcsolati beállítások, tokenek stb.

- **Konfiguráció betöltése és inicializálása:** Ez a réteg betölti az alkalmazás által használt beállításokat a megfelelő osztályokba, és inicializálja az alkalmazás konfigurációját.

Controller Package

Szerep: A Controller package felelős a bejövő HTTP kérések kezeléséért és azok feldolgozásáért.

Funkciók:

- **HTTP kérések fogadása:** A Controller réteg fogadja az érkező kéréseket a kliensektől.
- **Kérések feldolgozása:** A Controller osztályoknak általában módjuk van arra, hogy kinyerjék az adatokat a kérésből, és meghívják a megfelelő Service osztályokat a logika végrehajtáshoz.
- **Válaszok elküldése:** A Controller osztályok visszatérési értékei általában a HTTP válaszok, amelyeket a klienssel kommunikálva küldenek vissza.

Modell Package

Szerep: A Modell package a háttérbeli adatok reprezentálásáért felelős.

Funkciók:

- **Adatmodellek definiálása:** A Modell réteg tartalmazza azokat az osztályokat, amelyek reprezentálják az alkalmazásban használt különböző adatokat, például felhasználókat, foglaltságokat, autókat stb.
- **Adatok validálása és ellenőrzése:** A Model réteg gyakran tartalmaz validációs logikát az adatok megfelelőségének ellenőrzésére és biztosítására.

Service Package

Szerep: A Service package felelős a logika kezelésért és végrehajtásáért.

Funkciók:

- **Üzleti logika végrehajtása:** A Service réteg tartalmazza azokat az osztályokat, amelyek az alkalmazás logikáját végrehajtják, például adatok lekérdezése, manipuláció, szabályok érvényesítése stb.

- **Adatok kezelése és manipulálása:** A Service osztályok gyakran kapcsolatban állnak az adatbázissal vagy más adatforrásokkal, hogy adatokat lekérdezzenek, frissítsenek vagy töröljenek.

A projektben való rétegek felépítése

Config Package

CorsFilter osztály: Ez az osztály felelős a Cross-Origin Resource Sharing (CORS) beállítások kezeléséért a backend-en. Segít megakadályozni vagy engedélyezni a kereséseket különböző eredetű forrásokból.

Token osztály: Ez az osztály lehetőséget biztosít a tokenek kezelésére, például a JWT (JSON Web Token) generálására és ellenőrzésére.

Controller Package

- **ApplicationConfig:** A globális alkalmazásbeállításokat kezeli
- **UserController:** A felhasználókkal kapcsolatos műveletek irányítása.
- **RegisterController:** A regisztrációkat kezeli.
- **PaymentController:** A fizetésekkel kapcsolatos műveletek irányítása.
- **LoginController:** A bejelentkezések kérések fogadása.
- **CarServiceController:** Az autók szervizelésével kapcsolatos műveletek irányítása.
- **CarExtrasController:** Az autókhoz kapcsolatos extrák kezelése.
- **CarController:** Az autókkal kapcsolatos műveletek irányítása
- **CarAvailabilityController:** Az autók elérhetőségével kapcsolatos műveletek irányítása.
- **BookingsController:** A foglalásokkal kapcsolatos műveletek irányítása.
- **BookingXCarExtrasController:** A foglalások és ahhoz tartozó extrák kezelése.

Modell Package

- **User:** A felhasználókat reprezentáló modell.
- **Register:** A regisztrálókat reprezentáló modell.
- **Payment:** A fizetéseket reprezentáló modell.
- **Login:** A bejelentkezéseket reprezentáló modell.
- **Cars:** Az autókat reprezentáló modell.

- **CarService:** Az autók szervizelését reprezentáló modell.
- **CarExtras:** Az autókhoz tartozó extrákat reprezentáló modell.
- **CarAvailability:** Az autók elérhetőségét reprezentáló modell.
- **Bookings:** A foglalásokat reprezentáló modell.
- **BookingsXCarExtras:** A foglalások és ahhoz tartozó extrákat reprezentáló modell.

Service Package

- **UserService:** A felhasználókkal kapcsolatos függvények validációját tartalmazza.
- **RegisterService:** A regisztrációkkal kapcsolatos függvények validációját tartalmazza.
- **PaymentService:** A fizetésekkel kapcsolatos függvények validációját tartalmazza.
- **LoginService:** A bejelentkezéssel kapcsolatos függvények validációját tartalmazza.
- **CarService:** Az autók szervizelésével kapcsolatos függvények validációját tartalmazza.
- **CarExtrasService:** Az autókhoz tartozó extrákkal kapcsolatos függvények validációját tartalmazza.
- **CarAvailabilityService:** Az autók elérhetőségével kapcsolatos függvények validációját tartalmazza.
- **BookingsService:** A foglalásokkal kapcsolatos függvények validációját tartalmazza.

A frontend részletes leírása

Könyvtárstruktúra

A projektünk rendkívül rendezett és könnyen navigálható könyvtárstruktúrával rendelkezik. Az alapstruktúra a következő elemekből áll:

- **src:** Az alkalmazás forrásfájljait tartalmazza.
- **app:** Az alkalmazás főkönyvtára, amely tartalmazza az összes komponenst, szolgáltatást, modellt és a navigációk védelmét.
- **public:** Az alkalmazás statikus fájljait (pl.: képek) tartalmazza.

Komponensek

Az alkalmazás minden oldalát és funkcióját külön-külön komponensekre bontjuk:

login-page: Ez a komponens felelős a felhasználó bejelentkezési felületének megjelenítéséért és kezeléséért. Ide tartoznak a felhasználónevet és jelszót bekérő mezők, valamint a bejelentkezés gomb.

- Admin: Admin felületet biztosítja az adminok számára.
- Cars: Itt található az összes elérhető autó, amik közül lehet válogatni.
- Forgot-password: Jövőbeli fejlesztés...
- Home: Az weboldal nyitóoldala.
- Login: A felhasználó itt tud bejelentkezni.
- Navbar: Navigációs sáv, ahol elérhetőek az oldalhoz tartozó (About, Cars) navigációk.
- Profile-panel: Itt a felhasználó személyre tudja szabni a profilját.
- Register: A felhasználó itt tud bejelentkezni.
- Reservation: A felhasználó által lefoglalt járműveket vagy a jövőben lefoglalni kívánt járműveket jeleníti meg.

Stíluslapok

A projektünk stílusait kizárólag CSS nyelven írjuk, nem használunk preprocesszorokat. Minden komponens saját stílusfájllal rendelkezik, ami segít a stílusok elszigetelésében és azok könnyű kezelésében. A CSS szabályokat úgy alakítjuk ki, hogy a projekt egységes megjelenést mutasson, miközben a komponensek sajátos stílusjegyeiket is megőrzik. Emellett figyelünk arra is, hogy a stílusok ne legyenek túlterheltek vagy redundánsak, és csak azokat a szabályokat használjuk, amelyek valóban szükségesek a megfelelő megjelenés eléréséhez.

Reszponzív design

Az alkalmazásunkat úgy tervezzük, hogy responszív legyen, ami azt jelenti, hogy minden oldal és komponens rugalmasan alkalmazkodik a különböző kijelzőméretekhez és eszközökhöz.

AuthService

Az autentikáció és az autorizáció kezelését egy külön AuthService szolgáltatásban implementáljuk. Ez az osztály felelős az autentikációs és autorizációs logika megvalósításáért, valamint az endpointokhoz való hozzáférés kezeléséért.

AuthGuard: Az AuthGuard azért rendkívül előnyös, mert segít megvédeni a jogosulatlan hozzáférést a védett útvonalakhoz és komponensekhez. Ezáltal biztosítja, hogy csak a bejelentkezett és megfelelő jogosultságokkal rendelkező felhasználók érhessék el a kritikus funkciókat és adatokat, növelve a projekt biztonságát és a felhasználói élmény integritását.

HTTP kérések

Az endpointok kezelését az AuthService segítségével intézzük. Az HTTP kéréseket ezen a szolgáltatáson keresztül küldjük el a szerver felé, és az AuthService felelős a kérések kezeléséért és az azokra adott válaszok feldolgozásáért.

Modulok és szolgáltatások

Az alkalmazás különböző modulokra és szolgáltatásokra van osztva a funkciók és felelősségi területek szerint. Ezeket modulokat és szolgáltatásokat importáljuk és használjuk fel a megfelelő komponensekben, hogy biztosítsuk a szétválasztást és az egyszerűbb kezelhetőséget.

Angular modulok:

- **BrowserModule:** Az alapvető Angular modul, amely szükséges az Angular alkalmazások futtatásához böngészőkben.
- **CommonModule:** Ez a modul az angular egyik alapvető modulja. Ennek segítségével időt és erőfeszítést spórolhatunk meg
- **FormsModule:** Ez a modul segíti az űrlapok kezelését az Angular alkalmazásokban. Tartalmazza az ngModel direktívát és egyéb űrlapkezelő eszközöket.
- **RouterModule:** Irányelveket és szolgáltatókat ad hozzá az alkalmazáson belüli navigációhoz az alkalmazásban meghatározott nézetek között.
- **HttpClientModule:** Lehetővé teszi számunkra, hogy HTTP kéréseket hajtunk végre, és könnyen kezeljük ezeket a kéréseket és válaszaikat.

- **Observable:** Egy olyan objektum, amely idővel egy vagy több értéket bocsát ki. Létrehoz egy megfigyelhető példányt, amely szinkronban adja meg az argumentumként megadott értékeket.
- **OnInit:** Az összetevő konstruktorának végrehajtása és a komponens bemeneti tulajdonságainak inicializálása után kerül meghívásra.
- **Subscription:** Arra szolgál, hogy adatokat rögzítsen egy megfigyelő egy megfigyelhető előfizetés állapotáról.
- **Router:** Az útválasztás segít megváltoztatni, hogy a felhasználó mit lát egy egyoldalas alkalmazásban.
- **NgForm:** szögletes űrlappéldányt hoz létre, és űrlaphoz kapcsolódó funkciókat biztosít. Ezenkívül nyomon követi az űrlap érvényesítési állapotát, és szabályozza a benyújtás viselkedését.
- **AsyncPipe:** Feliratkozik egy megfigyelésre vagy ígéretre, és visszaadja az utoljára kiadott értéket. Amikor új érték kerül kiadásra, az megjelöli az ellenőrizni kívánt komponenst.
- **OnDestroy:** Életciklus hook, amelyet akkor hívnak meg, ha egy irányelv, pipe vagy szolgáltatás megsemmisül.

A frontend egyes részein lévő főbb funkciók

- A searchCarsBetweenDates függvény kilistázza az összes autót, ami a felhasználó által megadott időszámban van, így könnyedén ki tudja választani azokat az autókat, amik elérhetőek abban az időszámban.
- Az isUserExists függvény megnézi, hogy az adott felhasználó létezik-e, ha igen akkor nem enged még egy olyan felhasználót regisztrálni az oldalhoz.
- Az isAdminExists függvény megnézi, hogy az adott admin létezik-e, ha igen akkor nem enged még egy olyan admint regisztrálni az oldalhoz.
- A Login függvény a bejelentkezést vizsgálja, hogy helyes-e a megadott email, jelszó páros.
- A registerUser függvény a regisztrációkat jegyzi fel, ami alapján tud majd belépni a felhasználó.
- A registerAdmin függvény a regisztrációkat jegyzi fel, ami alapján tud majd belépni az admin.
- A getAllCarsPage függvény visszaadja az autókat, attól függően, hogy melyik oldalon találhatóak. Az első oldal 9 autót tartalmaz és a második oldal is szintén 9 autót tartalmaz.

Lokális szerver futtatása

Az alkalmazás teljes körű működését kiterjedt teszteléssel és futtatással alaposan ellenőriztük egy lokális 6 szerver környezetben. Ennek keretében a rendszer minden egyes komponensének, beleértve a frontend és backend funkcionálisokat, az adatbázis interakciókat, valamint a különböző szerverek közötti kapcsolatokat, alapos elemzésnek és validációnak vetettük alá. Az eredmények alapján biztosítottuk, hogy az alkalmazás minden része szerves egységet alkotva hatékonyan és összehangoltan működjön. A folyamatos tesztelés és a lokális 6 szerver környezetben történő egyidejű futtatás révén elérhettük, hogy az alkalmazás kiemelkedő teljesítményt nyújtson.

Csapatmunka

Szerepek

Timár Dávid vállalt szerepei és felelősségi területei a következők voltak a projekt során:

1. Frontend fejlesztő
 - A frontend részleg fejlesztése és kialakítása volt az ő feladata.
 - Megvalósította a felhasználói felület tervezését és implementálását.
 - Felelős volt az alkalmazás előtéri funkcionálisának kialakításáért és megvalósításáért.
2. Figma design terv készítése
 - A figma designterv megtervezése és megvalósítása volt az ő feladata.

Hederics Nikolasz vállalt szerepei és felelősségi területei a következők voltak a projekt során:

1. Backend fejlesztő

- Felelős volt a háttérrendszer kialakításáért és implementációjáért.
- Megvalósította a szerveroldali logikát és funkciókat.
- Biztosította, hogy a rendszer hatékonyan működjön.

2. Unit tesztelő

- Felelős volt a backend részegységeinek teszteléséért.
- Implementálta és futtatta a unit teszteket a kód minőségének és funkcionalitásának biztosítása érdekében.
- Ellenőrizte, hogy a backend kódja megfeleljen a tervezett specifikációknak és szabványoknak.

3. Manuális (Selenium) teszt

- A főbb funkciókat seleniumban csináltuk meg.

Lódrí Gergő vállalt szerepei és felelősségi területei a következők voltak a projekt során:

1. Project manager (Projektmenedzser)

- Feladata a projekt egészének irányítása és felügyelete.
- Koordinálta a csapatmunkát és ütemezte a fejlesztési folyamatokat.
- Hatékony kommunikáció biztosítása a csapattagok között és a project érintettjeivel.

2. Adatbázis fejlesztő

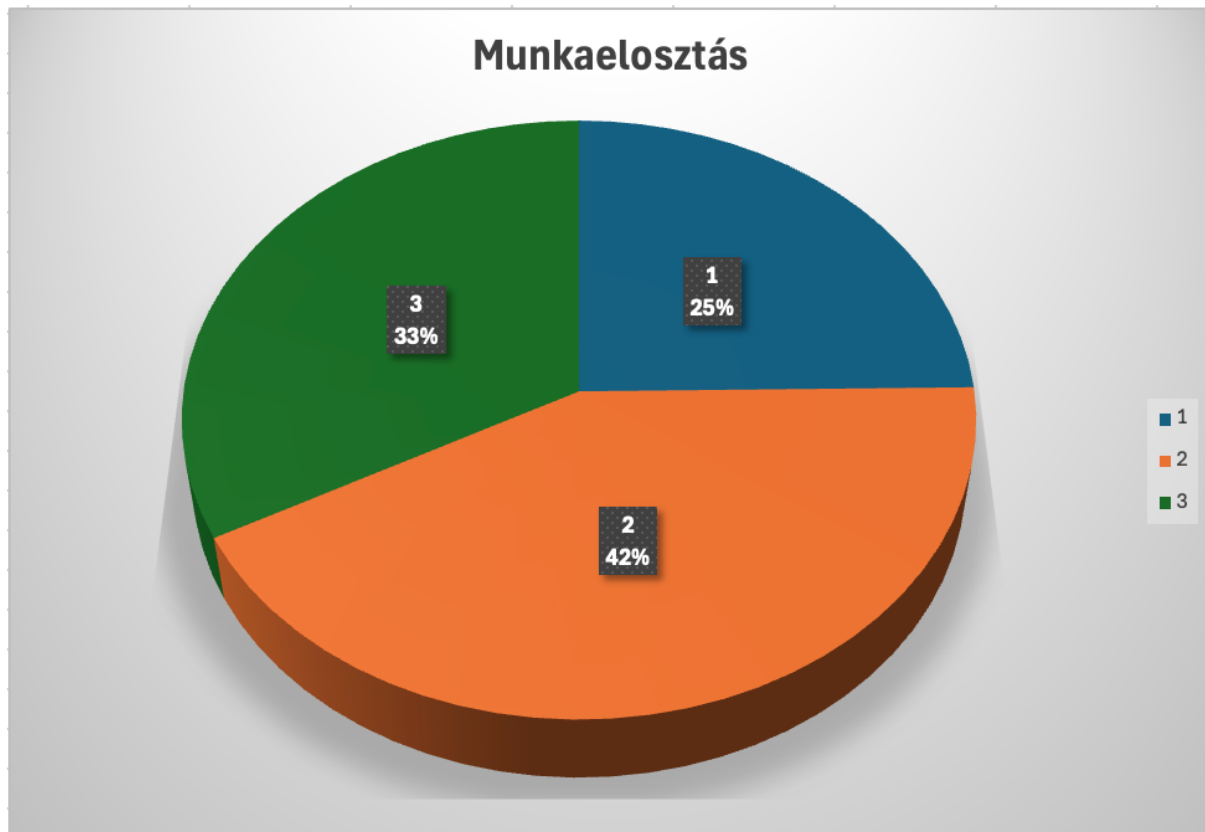
- Felelős volt az adatbázis kialakításáért és felépítéséért.
- Megvalósította az adatbázis struktúráját és kapcsolódási logikáját a rendszer számára.
- Biztosította az adatbázis hatékony működését és az adatok integritását a project specifikációinak megfelelően.

3. Backend segítő fejlesztő

- Felelős volt a háttérrendszer kialakításáért és implementációjáért.
- Megvalósította a szerveroldali logikát és funkciókat.
- Biztosította, hogy a rendszer hatékonyan működjön.

4. Manuális tesztelő

- Visszajelzéseket adott a fejlesztőcsapatnak a teszteredmények alapján.
- Biztosította, hogy az alkalmazás megfeleljen a minőségi követelményeknek.



Narancssárga: Timár Dávid

Kék: Hederics Nikolasz György

Zöld: Lódri Gergő

A projekt során szerzett tapasztalatok alapján megállapítottuk, hogy az agilis projektmenedzsment alkalmazása kiemelkedően hatékony volt rugalmas és hatékony fejlesztési folyamatok kialakításában. Az agilis megközelítés hasznosnak bizonyult a változó követelmények kezelésében, és a csapatmunka során szerzett tapasztalatok alapján azonosítottunk területeket, amelyeken a jövőben fejleszteni szeretnénk, például a projektmenedzsment finomhangolását.

Jövőbeli terveink

Kártyás fizetési lehetőség:

A jövőbeli terveink között szerepel, hogy az autót bérlő embereknek lehetőségük legyen online fizetni. Ezzel megkönnyíténénk és gyorsítanánk a foglalási folyamatot.

Üzenetküldő rendszer:

Az egyik legfontosabb fejlesztésünk egy teljesen új üzenetküldő rendszer bevezetése lesz. Ennek a rendszernek a célja az lesz, hogy modernizálja és optimalizálja a kommunikációt a felhasználóink között. Ez által az ügyfél meg tudja nézni, hogy mikorra és milyen autót foglalt le, hogy semmiképpen se felejtse el. Ez a fejlesztés tovább fokozza a platformunk által nyújtott értéket, javítva a felhasználói élményt és lehetőségeket.

Elfelejtetem a jelszót:

A program fejlesztése érdekében a jövőbeli terveink között szerepel az elfelejtetem a jelszót gomb, ami átirányít egy másik oldalra, ahol a felhasználó email segítségével meg tudja változtatni az elfelejtett jelszavát.

